

```
        return new FileJournal();
    }
}
```

Not every microservice needs to be a REST endpoint. In fact, we see more and more event-driven microservices these days. The *JournalService* is one such event-driven microservice. The following bootstrap is sufficient:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class UserJournalService {
    public static void main(String[] args) {
        SpringApplication.run(UserJournalService.class,
            args);
    }
}
```

The last step is to inject the configuration through *application.properties*:

```
spring.kafka.bootstrap-servers=kafka:9092
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-deserializer=org.apache.kafka.
common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=com.glarimy.ums.app.
UserDeserializer
ums.user.add.topic=com.glarimy.ums.user.add
ums.user.add.consumer=ums-consumer
```

Essentially, this configuration declares the topic on which the *JournalService* listens, the consumer group it belongs to, the deserializers it uses for extracting the keys and values and, of course, the Kafka broker it listens to.

The Dockerfile is simple:

```
FROM maven:3.5-jdk-8 AS build
COPY src /usr/glarimy/src
COPY pom.xml /usr/glarimy
RUN mvn -f /usr/glarimy/pom.xml clean package -DskipTests
ENTRYPOINT ["java","-jar","/usr/glarimy/target/ums-journal-
service.jar"]
```

Build the container image with the following command:

```
docker build -t glarimy/ums-journal-service.
```

And the following command publishes the image on the Docker hub:

```
docker push glarimy/ums-journal-service
```

Since the *JournalService* is not a user-facing service via REST API or any other means, we are not required to register it as a Kubernetes service. A simple deployment will work:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ums-journal-service
  labels:
    app: ums-journal-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ums-journal-service
  template:
```

```
krishnamohan_koyya@instance-1:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
kafka-588777fff6-jqt5r	1/1	Running	3 (29d ago)	51d
mysqldb-8454bb5bdd-dsqr6	1/1	Running	2 (29d ago)	51d
ums-add-service-7cb4dc9f9b-172tb	1/1	Running	2 (29d ago)	51d
ums-add-service-7cb4dc9f9b-rdbg8	1/1	Running	3 (29d ago)	51d
ums-add-service-7cb4dc9f9b-w2wzc	1/1	Running	2 (29d ago)	51d
ums-find-service-6dd7f56bb8-5gx7z	1/1	Running	2 (38m ago)	51d
ums-find-service-6dd7f56bb8-klfhd	1/1	Running	2 (38m ago)	51d
ums-find-service-6dd7f56bb8-klp4j	1/1	Running	2 (38m ago)	51d
ums-journal-service-5f7f8fb4d5-4tm4g	1/1	Running	0	14m
ums-search-service-5756d58799-7ntk2	1/1	Running	1 (38m ago)	29d
ums-search-service-5756d58799-s6b74	1/1	Running	1 (38m ago)	29d
ums-search-service-5756d58799-srqjn	1/1	Running	1 (38m ago)	29d
zookeeper-6bb6f5f5-1cv4r	1/1	Running	2 (29d ago)	51d

Figure 1: List of pods on the cluster